

Relatório de Atividade Prática: Representação de Grafos em Memória

Aluno: Gabriel Teotônio de Castro Coelho Costa

Matrícula: 885732

Estrutura de Dados Escolhida

Para garantir um armazenamento bom, não usei a Matriz de Adjacência que ocuparia um espaço de $O(n^2)$, tornando-se ruim para arquivos muito grandes. Em vez disso, usei a representação de Listas de Adjacência por meio de vetores.

Utilizei o Forward Star implementando dois vetores principais que ocupam um espaço proporcional a $O(n + m)$:

- **arc_dest:** armazena sequencialmente os vértices de destino de todas as arestas lidas.
- **pointer:** armazena os índices que demarcam onde começam e terminam as arestas de cada vértice de origem.

Lógica de Implementação e Ferramentas

- **Ferramentas Utilizadas:** O código foi escrito em Java, utilizando `BufferedReader` e `FileReader` para a leitura rápida e sequencial do arquivo de texto, e a classe `Scanner` para interagir com o usuário via terminal.
- **Construção da Estrutura:** O programa inicializa os vetores lendo a primeira linha do arquivo (que define o total de vértices e arestas). Durante a leitura, foi implementado um laço reverso para preencher os "buracos" no vetor `pointer`, tratando adequadamente os vértices isolados ou sem arestas de saída para evitar erros de índice.
- **Lógica de Consulta:** A busca pelo grau de saída e sucessores ocorre acessando diretamente o intervalo delimitado pelo vetor `pointer`. Para calcular o grau de entrada e listar os predecessores sem precisar gastar mais memória com uma estrutura *Reverse Star* adicional, aplicou-se uma varredura nas origens do grafo para filtrar quais vértices apontavam diretamente para o alvo.